Lista de Exercícios
Segunda prova - Procedimentos

Organização de Computadores

1 Cálculo do Mínimo Divisor Comum (MDC)

O Mínimo Divisor Comum (MDC) de dois números inteiros pode ser definido como o maior valor que é capaz de dividir ambos os números com resto zero. Por exemplo, o MDC dos números 15 e 12 é igual a 3, tendo em vista que $15 = 3 * 5$ e $12 = 2^2 * 3$. Quando dois números não possuem fatores primos em comum, temos que o MDC entre eles é igual a 1, uma vez que a divisão por 1 sempre tem resto zero.

O cálculo do MDC pode ser feito com o algoritmo de Euclides, descrito pelo pseudo-código a seguir:

```
Dados dois números a, b

Enquanto (a != 0 E b !=0):

    Se a > b:
        a ← resto de a/b
    Do contrário:
        b ← resto de b/a

Após sair do laço:

Se a = 0:
    O valor em b é o mdc

Se b = 0:
    O valor em a é o mdc
```

Item a) Mostre as iterações do algoritmo de Euclides para o par de números $(56, 21)$.

Item b) Escreva o código em Assembly x86 que implementa um procedimento que calcula o MDC entre os números armazenados em `AH` e `AL`, considerando ambos como inteiros de 8 bits sem sinal.

O resultado deverá ser armazenado no registrador AL.

2 Cálculo do Mínimo Múltiplo Comum (MMC)

O Mínimo Múltiplo Comum de dois números inteiros pode ser definido como o menor valor que pode ser dividido por ambos os números com resto igual a zero. Por exemplo, o MMC dos números 15 e 12 é igual a 60, uma vez que $60 = 15 * 2^2$ e $60 = 12 * 5$. Quando dois números não possuem fatores primos em comum, temos que o MMC deles é igual ao produto dos dois.

Podemos encontrar o MMC entre dois números por meio da relação:

$$\text{mmc } (a, b) = \frac{a * b}{\text{mdc } (a, b)}$$

Item a) Encontre o MMC do par de números $(56, 21)$.

Item b) Escreva o código em Assembly x86 que implementa um procedimento que calcula o MMC entre os números armazenados em `AH` e `AL` por meio da relação mostrada acima, considerando as entradas como inteiros de 8 bits sem sinal.

Suponha que o código já contém um procedimento chamado MDC que calcula o MDC entre os números armazenados em `AH` e `AL` e armazena o resultado no registrador `AL`, da mesma forma que o procedimento descrito no exercício anterior.

O resultado deverá ser armazenado no registrador `AX`.

3 Sequência de Fibonacci

A sequência de Fibonacci é uma sequência de números inteiros definida de forma recursiva, de modo que o valor de um termo é definido pela soma dos dois termos anteriores. Sendo F_n o n-ésimo termo da sequência, temos que:

$$F_n = F_{n-1} + F_{n-2}$$

Na sequência de Fibonacci, definimos $F_0 = 0$ e $F_1 = 1$.

Para além da definição recursiva da sequência de Fibonacci, é possível calcular o n-ésimo termo da sequência com o seguinte algoritmo:

```
Se N é 0 → Retorna 0
Se N é 1 → Retorna 1

Dados 3 números "anterior", "atual" e "proximo"

Contador ← 2

Enquanto Contador <= N:
    proximo ← atual + anterior
    anterior ← atual
    atual ← proximo
    Contador ← Contador + 1
```

`Retorna atual`

Item a) Calcule os termos da sequência de Fibonacci de F_0 até F_5.

Item b) Escreva o código em Assembly x86 que implementa um procedimento que calcula o n-ésimo termo da sequência de Fibonacci por meio do algoritmo descrito acima. Considere que o número N está armazenado no registrador `AL` e o resultado também deverá ser armazenado no registrador `AL`.

Item c) Escreva o código em Assembly x86 que implementa um procedimento que calcula o n-ésimo termo da sequência de Fibonacci por meio da definição recursiva dos termos da sequência. Considere que o número N está armazenado no registrador `AL` e o resultado também deverá ser armazenado no registrador `AL`.

Comece a sua solução com o pseudo-código a ser implementado.

Item d) Supondo que em um determinado programa a pilha foi definida com tamanho de $100h$ bytes, qual o maior valor de N que pode ser passado como argumento do procedimento de cálculo recursivo dos termos de Fibonacci sem que haja um estouro da pilha, isto é, sem que todo o espaço da pilha seja consumido?

Item e) Quantos bytes da pilha são ocupados por uma chamada de função?

4 Cálculo do Dígito Verificador de Luhn

Sequências de números de identificação costumam conter ao final os chamados dígitos verificadores, que são calculados a partir dos números anteriores aplicando-se alguma regra matemática.

Uma das técnicas de validação mais utilizadas é o algoritmo de Luhn, utilizado principalmente na verificação da validade dos números de cartões de crédito.

Dessa forma, iremos nesse exercício implementar um procedimento capaz de validar uma sequência de 16 dígitos correspondente a um possível cartão de crédito. Os 15 primeiros dígitos correspondem ao código de identificação daquele cartão, ao passo que o último dígito corresponde ao dígito verificador.

Utilizando o algoritmo de Luhn, o dígito verificador é calculado percorrendo os dígitos de identificação da direita para a esquerda, dobrando todo número em posição **ímpar** (primeiro, terceiro, quinto, etc).

Se o resultado da dobragem for maior que 9, subtrai-se 9 desse valor(o que equivale a somar os algarismos do resultado). Em seguida, somam-se todos os valores obtidos.

O dígito verificador é o menor valor $d \in \{0, \ldots, 9\}$ tal que a soma total acrescida de d seja múltipla de 10, isto é:

$$d = (10 - (\text{soma} \bmod 10)) \bmod 10.$$

Onde mod corresponde à operação módulo, ou seja, $a \bmod b$ é o resto da divisão de a por b.

O procedimento pode ser descrito pelo pseudo-código a seguir:

```
Dado um numero inteiro N sem sinal

soma ← 0
dobrar ← verdadeiro

Enquanto N != 0:
    digito ← N mod 10
    N ← N div 10

    Se dobrar = verdadeiro:
        digito ← 2 * digito
        Se digito > 9:
            digito ← digito - 9

    soma ← soma + digito
    dobrar ← nao dobrar

digito_verificador ← (10 - (soma mod 10)) mod 10
```

Item a) Mostre as iterações do algoritmo de Luhn para o número 7992.7398.718, calculando o seu dígito verificador.

Item b) Escreva o código em Assembly x86 que implementa um procedimento que calcula o dígito verificador de Luhn para uma string de 15 dígitos cujo endereço base está armazenado no registrador DX.

O dígito verificador, no intervalo de 0 a 9, deverá ser armazenado **como carácter ASCII** no registrador `AL`.